Unit 1 **Basics of Distributed Systems**

Introduction:

Examples of Distributed Systems

Trends in Distributed Systems

Focus on resource sharing

Challenges

System models:

Physical models

Architectural models

Fundamental model

1.1 WHAT IS DISTRIBUTED SYSTEM AND DISTRIBUTED COMPUTING?

- The process of computation was started from working on a single processor. This uni-processor computing can be termed as centralized computing.
- As the demand for the increased processing capability grew high, multiprocessor systems came to existence.
- The advent of multiprocessor systems led to the development of distributed systems with high degree of scalability and resource sharing.
- The modern-day parallel computing is a subset of distributed computing.
- A distributed system is a collection of independent computers, interconnected via a network, capable of collaborating on a task. Distributed computing is computing performed in a distributed system.
- In distributed systems, the entire network will be viewed as a computer. The multiple systems connected to the network will appear as a single system to the user. Thus, the distributed systems hide the complexity of the underlying architecture to the user.

The definition of distributed systems deals with two aspects that:

Deals with hardware: The machines linked in a distributed system are autonomous.

Deals with software: A distributed system gives an impression to the users that they are dealing with a single system.

Features of Distributed Systems:

- Communication is hidden from users.
- Applications interact in uniform and consistent way.
- High degree of scalability
- Resource sharing is possible in distributed systems.
- Distributed systems act as fault tolerant systems.
- Enhanced performance

Issues in distributed systems

- Concurrency - agreement
- Distributed system function in a heterogeneous environment. So, adaptability is a major issue.
- Memory considerations: The distributed systems work on both local and shared memory.
- Synchronization issues. They are less transparent.
- Since they are widespread, security is a major issue.
- Limits imposed on scalability.
- Knowledge about the dynamic network topology is a must.

QOS parameters

The distributed systems must offer the following QOS:

- Performance
- Reliability
- Availability
- Security

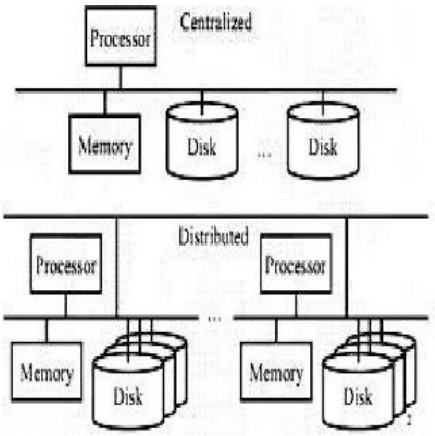

Fig 1.1: Distributed and Centralized systems

Differences between centralized and distributed systems

Centralized Systems	Distributed Systems
In Centralized Systems, several jobs are done on a particular central processing unit(CPU)	In Distributed Systems, jobs are distributed among several processors. The Processor are interconnected by a computer network
They have shared memory and shared variables.	They have no global state (i.e.) no shared memory and no shared variables.
Clocking is present.	No global clock.

1.2 SOME EXAMPLES OF DISTRIBUTED SYSTEMS

Today's internet exists on distributed systems. There are numerous applications of distributed systems.

Web Search

- The task of a web search engine is to index the entire contents of the World Wide Web.
- Distributed search is a search engine model in which the tasks of Web crawling, indexing and query processing are distributed among multiple computers and networks.
- The search engines were supported by a single supercomputer. But in recent years, they have moved to a distributed model.
- Google search relies upon thousands of computers crawling the Web from multiple locations all over the world.
- In Google's distributed search system, each computer involved in indexing crawls and reviews a portion of the Web, taking a URL and following every link available from it.
- The computer gathers the crawled results from the URLs and sends that information back to a centralized server in compressed format.
- The centralized server then coordinates that information in a database, along with information from other computers involved in indexing.
- The following features of Google search makes it act as distributed system:
  - The physical infrastructure with very large numbers of networked computers.
  - Highly distributed file system that supports very large files.
  - Availability of structured distributed storage system for fast access to data.
  - Distributed locking and agreement.
  - Works on a programming model that supports the management of large parallel and distributed computations.

- Selected application domains and associated networked applications
  - *Finance and commerce- Amazon, ebay, paypal*
  - *The information society- Wikipedia, Google, MySpace etc*
  - *Creative industries and entertainment- streaming content*
  - *Healthcare*-online electronic patient records and related issues, collaborative
  - working
  - *Education*-virtual learning environments
  - *Transport and logistics*- GPS, MapQuest, Google Maps and Google Earth
  - *Environmental management*- monitor and manage the natural environment,

Massively multiplayer online games (MMOGs)

- These games simulate real-life as much as possible. As such it is necessary to constantly evolve the game world using a set of laws.
- These Laws are a complex set of rules that the game engine applies with every clock tick.
- The virtual world consists not only of human players but also of all game elements that are not living objects.
- These elements are immutable and include the area terrain, trees, mountains, rivers, etc.
- MMOGs must be able to handle a very large number of simultaneous users.
- As the information transferred between the players and the game server is large, the bandwidth required to support a huge number of players if enormous.
- Very large virtual worlds require huge computational power to simulate the existence of life (AI Algorithms).
- These gaming applications work on both client server and distributed architectures.
- Therefore, both the bandwidth and computational load is spread out on many machines, thus making the application distributed.

Financial Trading

- Financial trading is now moving to distributed systems. These systems require frequent modifications, in response to the communication.
- Processing the events in distributed systems, demands reliability. So distributed event-based systems are used.
- A series of event feeds are given by the financial institution. The events may be in different formats, employ different technologies etc.
- So proper adaptors are a must. Pattern detection is also very import aspect in financial trading.
- The **Complex Event Processing (CEP)** is an automatized way of composing event together into logical, temporal or spatial patterns.

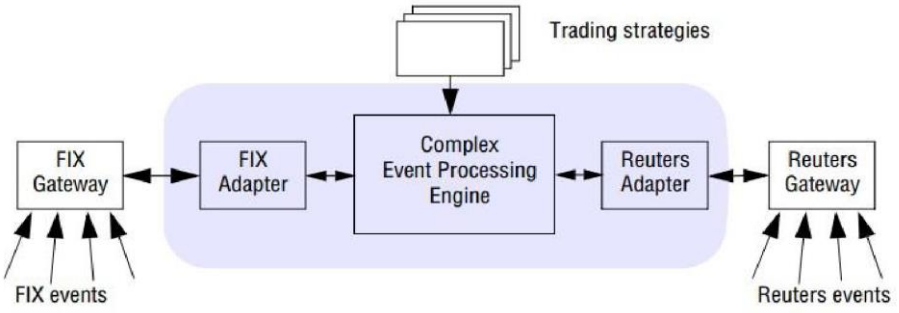

Fig 1.2: Financial Trading System

1.3 TRENDS IN DISTRIBUTED SYSTEMS

Distributed systems are undergoing a period of significant change and this can be traced back to a number of influential trends:

- the emergence of pervasive networking technology;
- the emergence of ubiquitous computing coupled with the desire to support user mobility in distributed systems;
- the increasing demand for multimedia services;
- the view of distributed systems as a utility.

1.3.1 Pervasive networking and the modern Internet

- Pervasive computing devices are completely connected and constantly available.
- The products that are connected to the pervasive network are easily available.
- The main goal of pervasive computing is to create an environment where the connectivity of devices is embedded in such a way that the connectivity is unobtrusive and always available.
- Internet can be seen as a large distributed system.

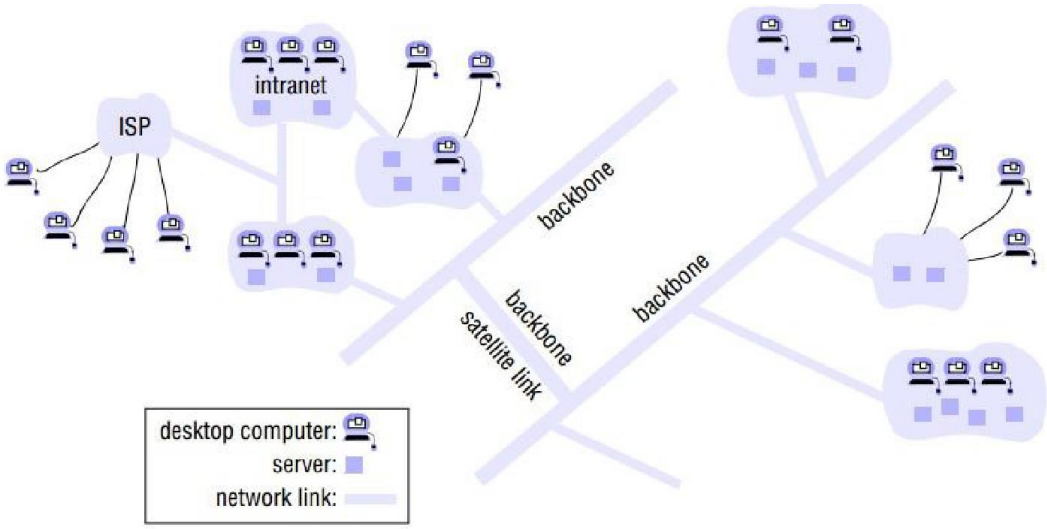

Fig 1.3: Internet

- The Internet is actually a collection of numerous sub networks operated by companies and other organizations and typically protected by firewalls.
- The role of a firewall is to protect an intranet by preventing unauthorized messages from leaving or entering.
- Internet Service Providers (ISPs) are companies that provide broadband links and other types of connection to individual users and small organizations, enabling them to access services anywhere in the Internet as well as providing local services such as email and web hosting.
- The intranets are linked together by backbones network link with a high transmission capacity, employing satellite connections, fibre optic cables and other high-bandwidth circuits.

1.3.2 Mobile and ubiquitous computing

- Internet is now built into numerous small devices from laptops to watches.
- These devices must have a high degree of portability. Mobile computing supports this.
- Mobile computing is the performance of computing tasks while the user dynamically changes his geographic location.

- Ubiquitous computing (small computing) means that all small computing devices will eventually become so pervasive in everyday objects.

Differences between ubiquitous computing and mobile computing

Ubiquitous Computing	Mobile Computing
They could connect tens/hundreds of computing devices in every room/person, becoming "invisible" and part of the environment – WANs, LANs, PANs – networking in small spaces	They could connect a few devices for every person, small enough to carry around – devices connected to cellular networks or WLANs
They could connect even the non- mobile devices and offer various forms of communication.	They are actually a subset of ubiquitous computing.
They could support all forms of devices that are connected to the internet from laptops to watches.	They support only conventional, discrete computers and devices.

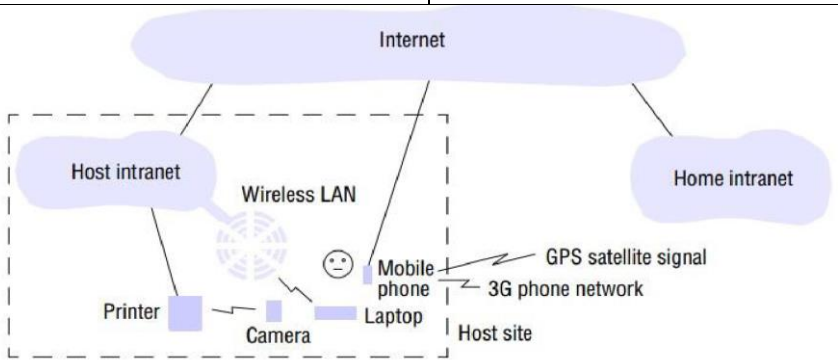

Fig 1.4: Mobile devices in an environment

- The user in the above given environment has access to three forms of wireless connection:
  - ✓ The laptop connects to the host's wireless LAN.
  - ✓ Mobile (cellular) telephone connected to the Internet. The phone gives access to the Web and other Internet services.
  - ✓ Digital camera that communicates over a personal area wireless network.
- This scenario demands associations between devices are routinely created and destroyed, called **spontaneous interoperation**. This interoperation must be fast and convenient.

1.3.3 Distributed multimedia systems

- A distributed system supports the storage, transmission and presentation of discrete media types.
- A distributed multimedia system should be able to perform the same functions for continuous media types such as audio and video.
- It should be able to store and locate audio or video files, to transmit them across the network to support the presentation of the media types to the user and optionally also to share the media types across a group of users.
- The processing of such media files includes handling of temporal dimensions and integrity of media.
- This includes access to live or pre-recorded television broadcasts, access to film libraries offering video-on-demand services, access to music libraries, the provision of audio and video conferencing facilities and integrated telephony features including IP telephony or related technologies such as Skype, a peer-to- peer alternative to IP telephony.

- Webcasting is an application of distributed multimedia technology. Webcasting is the ability to broadcast continuous media, typically audio or video, over the Internet.
- Webcasting demands the following changes in the infrastructure:
    - Support for a range of encoding and encryption formats.
    - Support for a range of mechanisms to ensure that the desired quality of service can be met.
    - Adaptability to associated resource management strategies.
    - Providing adaptation strategies to deal with the situation where QoS is difficult to achieve.

1.3.4 Distributed computing as a utility

- Distributed systems are seen as a utility like lectricity.
- The resources are provided by appropriate service suppliers and effectively rented by the end user.
- The services may be physical or logical services.
- Physical resources such as storage and processing can be made available to networked computers through data centers.
- Operating system virtualization is a key enabling technology that users may actually be provided with services by a virtual rather than a physical node.
- Software services rental, services such as email and distributed calendars.
- Cloud computing is the child of resource sharing.
- A cloud is defined as a set of Internet-based application, storage and computing services sufficient to support most users' needs, thus enabling them to largely or totally dispense with local data storage and application software with local data storage and application software.
- Clouds are generally implemented on cluster computers.
- A cluster computer is a set of interconnected computers that cooperate closely to provide a single, integrated high performance computing capability.

1.4 RESOURCE SHARING

- An important goal of a distributed system is to effectively utilize the collective resources of the system, namely, the memory and the processors of the individual nodes.
- Users think in terms of shared resources such as a search engine without regard for the server or servers that provide these.
- A search engine on the Web provides a facility to users throughout the world, users who need never come into contact with one another directly.
- In computer-supported cooperative working (CSCW), a group of users who cooperate directly share resources such as documents in a small, closed group.
- The pattern of sharing and the geographic distribution of particular users determine what mechanisms the system must supply to coordinate users' actions.
- Resources in a distributed system are physically encapsulated within computers and can only be accessed from other computers by means of communication.
- Sharing is done by a program that offers a communication interface enabling the resource to be accessed and updated reliably and consistently.
- In a client server paradigm, server refers to a running program on a networked computer

that accepts requests from programs running on other computers to perform a service and responds appropriately.

- The requesting processes are referred to as clients.
- A complete interaction between a client and a server, from the point when the client sends its request to when it receives the server's response, is called a remote invocation.

1.5 CHALLENGES IN DISTRIBUTED SYSTEMS

a) Heterogeneity

Heterogeneity means the diversity of the distributed systems in terms of hardware, software, platform, etc. Modern distributed systems will likely to be operating with different:

- Hardware devices: computers, tablets, mobile phones, embedded devices, etc.
- Operating System: Ms Windows, Linux, Mac, Unix, etc.
- Network: Local network, the Internet, wireless network, satellite links, etc.
- Programming languages: Java, C/C++, Python, PHP, etc.
- Different roles of software developers, designers, system managers
- Middleware: Middleware applies to a software layer that provides a programming abstraction as well as masking the heterogeneity of the underlying networks, hardware, operating systems and programming languages. Eg: CORBA, RMI.
- Heterogeneity in mobile code: Mobile code is used to refer to program code that can be transferred from one computer to another and run at the destination.

Eg: Java applets.

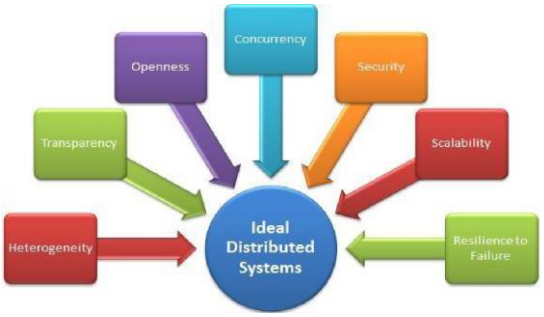

Fig 1.5: Challenges in Distributed systems

b) Transparency

Some terms of transparency in distributed systems are:

Transparency	Description
Access	Hide differences in data representation and how a resource is accessed
Location	Hide where a resource is located
Migration	Hide that a resource may move to another location
Relocation	Hide that a resource may be moved to another location while in use
Replication	Hide that a resource may be copied in several places
Concurrency	Hide that a resource may be shared by several competitive users
Failure	Hide the failure and recovery of a resource
Persistence	Hide whether a (software) resource is in memory or a disk

The access and location transparency is collectively referred as **network transparency.**

c) Openness

If the well-defined interfaces for a system are published, it is easier for developers to add new features or replace sub-systems in the future. Example: Twitter and Facebook have API that allows developers to develop their software. The following are key points in openness:

- key interfaces are published.
- uniform communication mechanism and published interfaces for access to shared resources.
- Open distributed systems can be constructed from heterogeneous hardware and software, possibly from different vendors.

d) Concurrency

Distributed Systems usually is multi-users environment. In order to maximize concurrency, resource handling components should anticipate as they will be accessed by competing users. Concurrency prevents the system from becoming unstable when users compete to view or update data.

e) Security

Every system must consider strong security measurements. The following attacks are more common in distributed systems:

Denial of service attacks: When the requested service is not available at the time of request it is Denial of Service (DOS) attack. This attack is done by bombarding the service with a large number of useless requests that the serious users are unable to use it.

Security of mobile code: Mobile code needs to be handled with care since they are transmitted in an open environment.

f) Scalability

Distributed systems must be scalable as the number of user increases. A system is said to be scalable if it can handle the addition of users and resources without suffering a noticeable loss of performance or increase in administrative complexity.

Scalability has 3 dimensions:

Size: Number of users and resources to be processed. Problem associated with this is overloading.

Geography: Distance between users and resources. Problem associated is communication reliability

Administration: As the size of distributed systems increases, many of the systems need to be controlled. Problem associated is administrative mess

Scalability often conflicts with small system performance. Claims of scalability in such systems are often abused. The following are the scalability challenges:

- Controlling the cost of physical resources
- Controlling the performance loss
- Preventing software resources running out
- Avoiding performance bottlenecks

g) Resilience to Failure (Fault tolerance)

Distributed Systems involves a lot of collaborating components (hardware, software, communication). So there is a huge possibility of partial or total failure. The failures are handled in series of steps:

Detecting failures: Some failures like checksum can be detected.

Masking failures: Some failures that have been detected can be hidden or made less severe. Examples of hiding failures include retransmission of messages and maintaining a redundant copy of the same data.
Tolerating failures: All the failures cannot be handled. Some failures must be accepted by the user. Example of this is waiting for a video file to be streamed in.
Recovery from failures: Recovery involves the design of software so that the state of permanent data can be recovered or rolled back after a server has crashed.
Redundancy: Services can be made to tolerate failures by the use of redundant components. Examples for this include: maintenance of two different paths between the same source and destination.
Availability is also a major concern in fault tolerance. The **availability of** a system is a measure of the proportion of time that it is available for use. It is a useful performance metric.
h) Quality of Service:
The distributed systems must confirm the following non functional requirements:
Reliability: A reliable distributed system is designed to be as fault tolerant as possible. Reliability is the quality of a measurement indicating the degree to which the measure is consistent.
Security: Security is the degree of resistance to, or protection from, harm. It applies to any vulnerable and valuable asset, such as a person, dwelling, community, nation, or organization. Distributed systems spread across wide geographic locations. So security is a major concern.
Adaptability: The frequent changing of configurations and resource availability demands the distributed system to be highly adaptable.

1.6 SYSTEM MODELS

System models describe common properties and design choices for distributed systems in a single descriptive model. The system models of distributed systems are classified into the following types:
Physical models: It is the most explicit way in which to describe a system in terms of hardware composition.
Architectural models: They describe a system in terms of the computational and communication tasks performed by its computational elements.
Fundamental models: They examine individual aspects of a distributed system.
They are again classified based on some parameters as follows:

- **Interaction models:** This deals with the structure and sequencing of the communication between the elements of the system
- **Failure models:** This deals with the ways in which a system may fail to operate correctly
- **Security models:** This deals with the security measures implemented in the system against attempts to interfere with its correct operation or to steal its data.

1.6.1 Physical Models

- A physical model is a representation of the underlying hardware elements of a distributed system that hides the details of the computer and networking technologies employed.
- There are many physical models available from the primitive baseline model to the complex models that could handle cloud environments.

Baseline physical model:

- This is a primitive model that describes the hardware or software components located in networked computers.

- The communication and coordination of their activities is done by passing messages.
- This is a minimal physical model of a distributed system.
- This model could be extended to a set of computer nodes interconnected by a computer network for the required passing of messages.
- This model helps us to categorize the three generations of distributed systems.

Early distributed systems:

- The period is in the late 1970s and early 1980s.This was developed after the emergence of LAN.These systems could support 10 to 100 nodes interconnected by a LAN.
- It has very limited internet connectivity and can support a small range of services such as shared local printers and file servers.
- Individual systems were homogeneous, with poor quality of service.

Internet-scale distributed systems:

- The period is from the 1990s after the growth of internet.
- The physical set up of distributed system is described as an extensible set of nodes interconnected by a network of networks (the Internet).
- They incorporate large numbers of nodes and provide distributed system services.
- The systems were heterogeneous which could adopt open standards.
- They had good QOS.

Contemporary distributed systems:

- The nodes were desktop computers and therefore relatively static, discrete and independent.
- Mobile computing has led to physical models with movable nodes. Service discovery is of primary concern in these systems.
- The cloud computing and cluster architectures hassled to pools of nodes that collectively provide a given service. This resulted in enormous number of systems given the service.

Distributed systems of systems:

- The ultra large scale (ULS) distributed systems is defined as a complex system consisting of a series of subsystems that are systems in their own right and that come together to perform a particular task or tasks. Some important characteristics of these systems include their
    - very large size
    - global geographical distribution
    - operational and managerial independence of their member systems.

The main function of these systems arises from the interoperability between their components.

Parameter	Early Systems	Internet Scale	ULS System
Scale	Small	Large	Ultra Large
Heterogeneity	Homogeneous	Platform independent software and technologies.	Supports many types of architectures
Openness	Systems are mostly closed	Many open standards are available	The already existing standards are not able to cope with complex systems.
QOS	Poor	Significant	QOS could still be improved

1.6.2 Architectural Models

- The architecture abstracts the functions of the individual components of the distributed system.
- This describes the components and their interrelationships to ensure the system is reliable, manageable, adaptable and cost-effective. The different models are evaluated based on the following criteria:
  - architectural elements
  - architectural patterns
  - middleware platforms

1.6.2.1 Architectural elements

The following must be decided to build a distributed system:

- Communicating entities (objects, components and web services);
- Communication paradigms (inter process communication, remote invocation and indirect communication);
- Roles and responsibilities and placement

1.6.2.1.1 Communicating entities

The components that are communicating and how those entities communicate together are dealt here. The following are some of the problem-oriented entities:

Objects: They are used to implement object-oriented approaches in distributed systems. Objects are accessed through interfaces.

Object	Components	Web services
Object oriented solution	Problem oriented solution	Problem oriented solution
The dependencies and assumptions of the interfaces holds only on the objects that act upon them.	The dependencies and assumptions of the all interfaces holds on all the components in the system.	They are integrated into the WWW.
Tightly coupled services	Tightly coupled services	Complete service that could also be made as a value added service

Components: Components resemble objects and they offer problem-oriented abstractions for building distributed systems. They are also accessed through interfaces. The key difference between component and object is that a components holds all the assumptions specified to other components and their interfaces in the system. Components and objects are used to develop tightly coupled applications.

Web services: Web services are closely related to objects and components. Web services are integrated into the World Wide Web. They are partially defined by the web-based technologies they adopt. Web services are generally viewed as complete services that can be combined to achieve value-added services across organizational boundaries.

1.6.2.1.2 Communication paradigms

There are three major modes of communication in distributed systems:

- **Interprocess communication**:
  This is a low-level support for communication between processes in distributed systems, including message-passing primitives. They have direct access to the API offered by Internet protocols and support multicast communication.
- **Remote invocation:**
  It is used in a distributed system and it is the calling of a remote operation, procedure or method.

a) **Request-reply protocols:**
This is a message exchange pattern in which a requestor sends a request message to a replier system which receives and processes the request, ultimately returning a message in response. This is a simple, but powerful messaging pattern which allows two applications to have a two-way conversation with one another over a channel. This pattern is especially common in client–server architectures.
Remote procedure calls: Remote Procedure Call (**RPC**) is a protocol that one program can use to request a service from a program located in another computer in a network without having to understand network details.

b) **Remote method invocation**: RMI (Remote Method Invocation) is a way that a programmer can write object-oriented programming in which objects on different computers can interact in a distributed network. This has the following two key features:

- ✓ Space uncoupling: Senders do not need to know who they are sending to
- ✓ Time uncoupling: Senders and receivers do not need to exist at the same time

➢ **Indirect communication**

Key techniques for indirect communication include:

a) **Group communication:** Group communication is concerned with the delivery of messages to a set of recipients and hence is a multiparty communication paradigm supporting one-to-many communication. A group identifier uniquely identifies the group. The recipients join the group and receive the messages. Senders send messages to the group based on the group identifier and hence do not need to know the recipients of the message.

b) **Publish-subscribe systems:** This is a messaging pattern where senders of messages, called publishers, do not program the messages to be sent directly to specific receivers, called subscribers. Instead, published messages are characterized into classes, without knowledge of what, subscribers there may be. Similarly, subscribers' express interest in one or more classes, and only receive messages that are of interest, without knowledge of what, if any, publishers there are. They offer one-to-many style of communication.

c) **Message queues:** They offer a point-to-point service whereby producer processes can send messages to a specified queue and consumer processes can receive messages from the queue or be notified of the arrival of new messages in the queue. Queues therefore offer an indirection between the producer and consumer processes.

d) **Tuple spaces:** The processes can place arbitrary items of structured data, called tuples, in a persistent tuple space and other processes can either read or remove such tuples from the tuple space by specifying patterns of interest. This style of programming is known as generative communication.

e) **Distributed shared memory:** In computer architecture, distributed shared memory (DSM) is a form of memory architecture where the (physically separate) memories can be addressed as one (logically shared) address space. Here, the term shared does not mean that there is a single centralized memory but shared essentially means that the address space is shared (same physical address on two processors refers to the same location in memory.

1.6.2.1.3 Roles and responsibilities

There are two types of architectures based on roles and responsibilities:

1) **Client-server:**

The system is structured as a set of processes, called servers, that offer services to the users, called clients.
The client-server model is usually based on a simple request/reply protocol, implemented with send/receive primitives or using remote procedure calls (RPC) or remote method invocation (RMI).

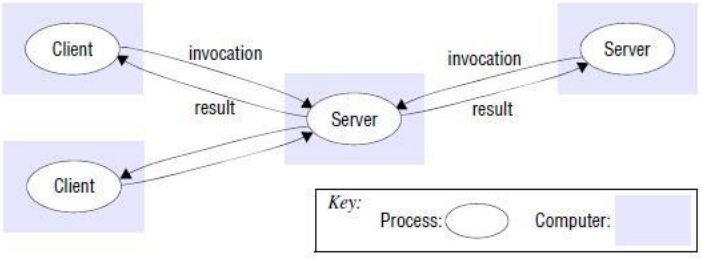

Figure 2.1: Client server style

2) Peer-peer

- All processes (objects) play similar role. Processes (objects) interact without particular distinction between clients and servers.
- The pattern of communication depends on the particular application.
- A large number of data objects are shared; any individual computer holds only a small part of the application database.

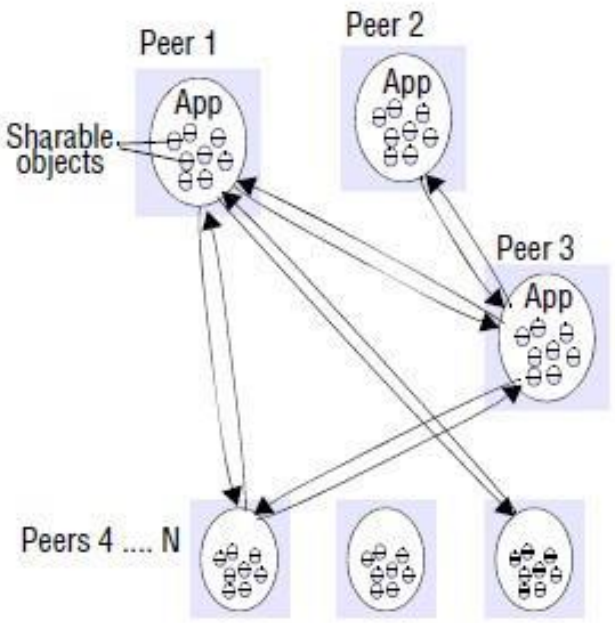

Figure 2.2: Peer-peer style

- Peer-to-Peer model distributes shared resources widely share computing and communication loads.
- Problems with peer-to-peer are high complexity due to cleverly place individual objects and there are large number of replicas.

3. Placements:

Mapping of objects or services to the underlying physical distributed infrastructure is considered here. The following points must be taken care while placing the objects: reliability, communication pattern, load, QOS etc. The following are the placement strategies:

Mapping of services to multiple servers

- The servers may partition the set of objects on which the service is based and distribute those objects between themselves, or they may maintain replicated copies of them on several hosts.
- The Web provides a common example of partitioned data in which each web server manages its own set of resources. A user can employ a browser to access a resource at any one of the servers.

Caching

- A cache is a local store of recently used data objects that is closer to one client or a particular set of clients than the objects themselves.

- When a new object is received from a server it is added to the local cache store, replacing some existing objects if necessary.
- When an object is needed by a client process, the caching service first checks the cache and supplies the object from there if an up-to-date copy is available. If not, an up-to-date copy is fetched.
- Caches may be co-located with each client or they may be located in a proxy server that can be shared by several clients.

Mobile code:

- Applets are a well-known and widely used example of mobile code. The user running a browser selects a link to an applet whose code is stored on a web server; the code is downloaded to the browser and runs. They provide interactive response.

Mobile agents:

- A mobile agent is a running program (including both code and data) that travels from one computer to another in a network carrying out a task on someone‟s behalf, such as collecting information, and eventually returning with the results.
- A mobile agent is a complete program, code + data, that can work (relatively) independently. The mobile agent can invoke local resources/data.
- Typical tasks of mobile agent includes: collect information , install/maintain software on computers, compare prices from various vendors bay visiting their sites etc.

1.6.2.2 Architectural patterns

Architectural patterns are reusable solution to a commonly occurring problem in software architecture within a given context. The following are some of the common architectural patterns:

Layerings:

In a layered approach, a complex system is partitioned into a number of layers, with a given layer making use of the services offered by the layer below.

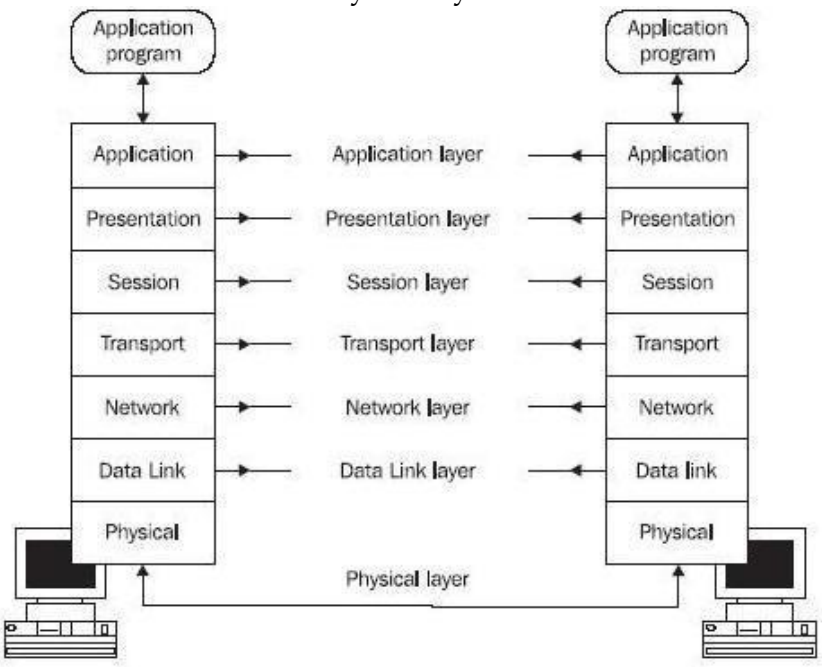

Figure 2.3: Layered Architecture

- A given layer is unaware of the implementation details, of other layers beneath them.
- In terms of distributed systems, this equates to a vertical organization of services into service layers.
- A distributed service can be provided by one or more server processes, interacting with each other and with client processes in order to maintain a consistent system-wide view of the service‟s resources.

Tiered architecture

- Tiering is a technique to organize functionality of a given layer and place this functionality into appropriate servers and, as a secondary consideration, on to physical nodes.
- The two tiered architecture refers to client/server architectures in which the user interface (presentation layer) runs on the client and the database (data layer) is stored on the server. The actual application logic can run on either the client or the server.

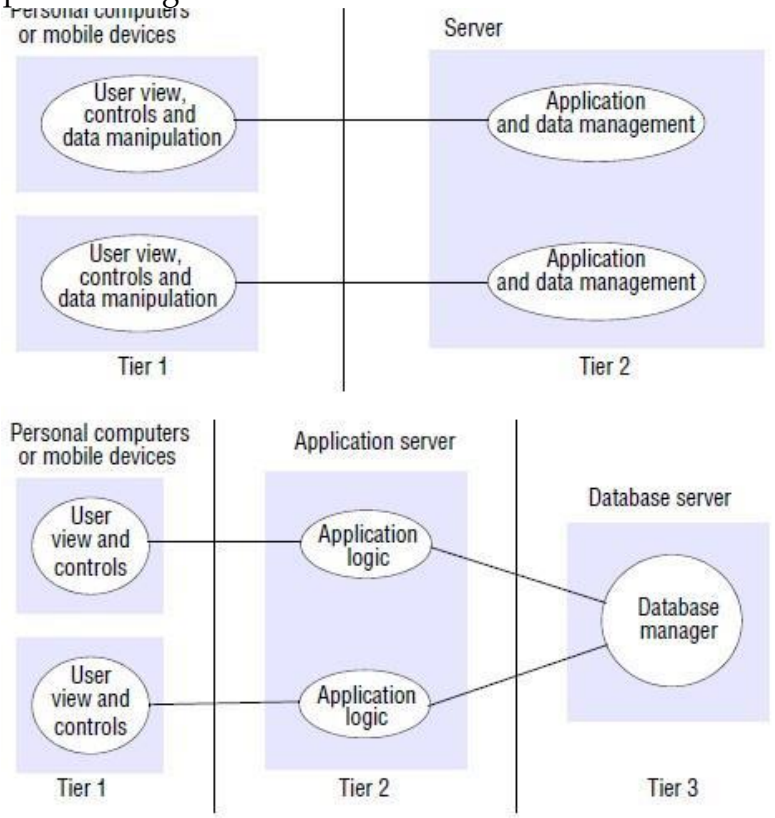

Fig 2.4: Two and three tiered architectures

The three-tiered architecture has:

a) Presentation tier: This is the topmost level of the application. is concerned with handling user interaction and updating the view of the application as presented to the user;

b) Application tier (business logic, logic tier, or middle tier): The logical tier is pulled out from the presentation tier and, as its own layer, it controls an application‟s functionality by performing detailed processing.

c) Data tier: The data tier includes the data persistence mechanisms (database servers, file shares, etc.) and the data access layer that encapsulates the persistence mechanisms and exposes the data.

Thin Clients:

- In a thin-client model, all of the application processing and data management is carried out on the server.
- The major disadvantages of thin clients is that it places a heavy processing load on both the server and the network and the delay due to operating system latencies.
- The virtual network computing (VNC) has emerged to overcome the disadvantages of thin clients.
- VNC is a type of remote-control software that makes it possible to control another computer over a network connection.
- Since all the application data and code is stored by a file server, the users may migrate from one network computer to another. So VNC is of big use in distributed systems.

Other patterns

a) Proxy:

- This facilitates location transparency in remote procedure calls or remote method invocation.
- A proxy is created in the local address space to represent the remote object with same interface as the remote object.
- The programmer makes calls on this proxy object and he need not be aware of the distributed nature of the interaction.

b) Brokerage:

- It is used to bring interoperability in potentially complex distributed infrastructures.
- The service broker is meant to be a registry of services, and stores information about what services are available and who may use them.

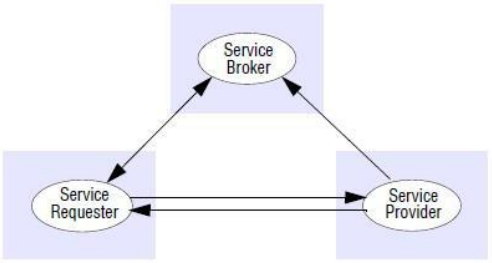

Fig 2.5: Web service with brokers

c) Reflection:

- The Reflection architectural pattern provides a mechanism for changing structure and behavior of software systems dynamically.
- It supports the modification of fundamental aspects, such as type structures and function call mechanisms.
- In this pattern, an application is split into two parts:

1) **Meta Level:** This provides information about selected system properties and makes the software self-aware.

2) **Base level:** This includes the application logic. Its implementation builds on the meta level. Changes to information kept in the meta level affect subsequent base-level behavior.

1.6.2.3 Associated middleware solutions

- Middleware provides a higher-level programming abstraction for the development of distributed systems.
- Middleware makes it easier for software developers to perform communication and input/output, so they can focus on the specific purpose of their application.
- Middleware is the software that connects software components or enterprise applications and it lies between the operating system and the applications on each side of a distributed computer network.
- Middleware includes Web servers, application servers, content management systems, and similar tools that support application development and delivery.

Categories of middleware

The following are some of the categories of middleware:

Distributed Objects

The term distributed objects usually refers to software modules that are designed to work together, but reside either in multiple computers connected via a network or in different processes inside the same computer. One object sends a message to another object in a remote machine or process to perform some task. The results are sent back to the calling object.

Distributed Components

A component is a reusable program building block that can be combined with other components in the same or other computers in a distributed network to form an application.

Examples: a single button in a graphical user interface, a small interest calculator, an interface to a database manager.
Components can be deployed on different servers in a network and communicate with each other for needed services. A component runs within a context called a container.
Examples: pages on a Web site, Web browsers, and word processors.

Publish subscriber model
Publish–subscribe is a messaging pattern where senders of messages, called publishers, do not program the messages to be sent directly to specific receivers, called subscribers. Instead, published messages are characterized into classes, without knowledge of what, if any, subscribers there may be. Similarly, subscribers express interest in one or more classes, and only receive messages that are of interest, without knowledge of what publishers are there.

Message Queues
Message queues provide an asynchronous communications protocol, meaning that the sender and receiver of the message do not need to interact with the message queue at the same time. Messages placed onto the queue are stored until the recipient retrieves them. Message queues have implicit or explicit limits on the size of data that may be transmitted in a single message and the number of messages that may remain outstanding on the queue.

Webservices
A web service is a collection of open protocols and standards used for exchanging data between applications or systems. Software applications written in various programming languages and running on various platforms can use web services to exchange data over computer networks like the Internet in a manner similar to inter-process communication on a single computer. This interoperability (e.g., between Java and Python, or Windows and Linux applications) is due to the use of open standards.

Peer to peer
Peer-to-peer (P2P) computing or networking is a distributed application architecture that partitions tasks or work load between peers. Peers are equally privileged, equipotent participants in the application. They are said to form a peer- to-peer network of nodes.

Advantages of middleware
- Real time information access among systems.
- Streamlines business processes and helps raise organizational efficiency.
- Maintains information integrity across multiple systems.
- It covers a wide range of software systems, including distributed Objects and components, message-oriented communication, and mobile application support.
- Middleware is anything that helps developers create networked applications.

Disadvantages of middleware
- Prohibitively high development costs.
- There are only few people with experience in the market place to develop and use a middleware.
- There are only few satisfying standards.
- The tools are not good enough.
- Too many platforms to be covered.
- Middleware often threatens the real-time performance of a system.
- Middleware products are not very mature.

1.6.3 Fundamental Models

The purpose of the fundamental model is to make explicit all the relevant assumptions about the systems we are modelling and to make generalizations (general purpose algorithms) concerning what is possible or impossible with given assumptions.

They are of **three types**

Interaction Model – deals with performance and the difficulty of setting of time limits in a distributed system.

Failure Model – specification of the faults that can be exhibited by processes.

Secure Model – discusses possible threats to processes and communication channels.

1.6.3.1 Interaction model

In this model the computations occur within processes. The processes interact by passing messages to achieve communication (information flow) and coordination (synchronization and ordering of activities) between processes. The two significant factors affecting interacting processes in a distributed system are:

1) Communication performance

2) Global notion of time.

Communication Performance

The communication channel in can be implemented in a variety of ways like Streams or through simple message passing over a network. The performance characteristics of a network are:

a) **Latency:** A delay between the start of a message‟s transmission from one process to the beginning of reception by another.

b) **Bandwidth:** The total amount of information that can be transmitted over in a given time. The communication channels using the same network, have to share the available bandwidth.

c) Jitter: The variation in the time taken to deliver a series of messages. It is very relevant to multimedia data.

Global Notion of time

- Each computer in a distributed system has its own internal clock, which can be used by local processes to obtain the value of the current time.
- Any two processes running on different computers can associate timestamp with their events.
- However, even if two processes read their clocks at the same time, their local clocks may supply different time because the computer clock drifts from perfect time and their drift rates differ from one another.
- Even if the clocks on all the computers in a DS are set to the same time initially, their clocks would eventually vary quite significantly unless corrections are applied.
- There are several techniques to correct time on computer clocks. One technique is to use radio receivers to get readings from GPS (Global Positioning System) with accuracy about 1 microsecond.

Variants of Interaction model

- In a DS it is hard to set time limits on the time taken for process execution, message delivery or clock drift. There are two models based on time stamp:

➢ **Synchronous DS**

- This is practically hard to achieve in real life. In synchronous DS the time taken to execute a step of a process has known lower and upper bounds.
- Each message transmitted over a channel is received within a known bounded time. Each process has a local clock whose drift rate from real time has known bound.

➢ **Asynchronous DS**

- There are no bounds on process execution speeds, message transmission delays and clock drift rates.

Event Modeling

- Event ordering is of major concern in DS. The concept of one event happening before another in a distributed system is examined, and is shown to define a partial ordering of the events.
- The execution of a system can be described in terms of events and their ordering despite the lack of accurate clocks. Consider a mailing list with users X, Y, Z, and A.
- Due to independent delivery in message delivery, message may be delivered in different order.
- If messages m1, m2, m3 carry their time t1, t2, t3, then they can be displayed to users accordingly to their time ordering.
- The mail box is:

Item	From	Subject
23	Z	Re: Meeting
24	X	Meeting
26	Y	Re: Meeting

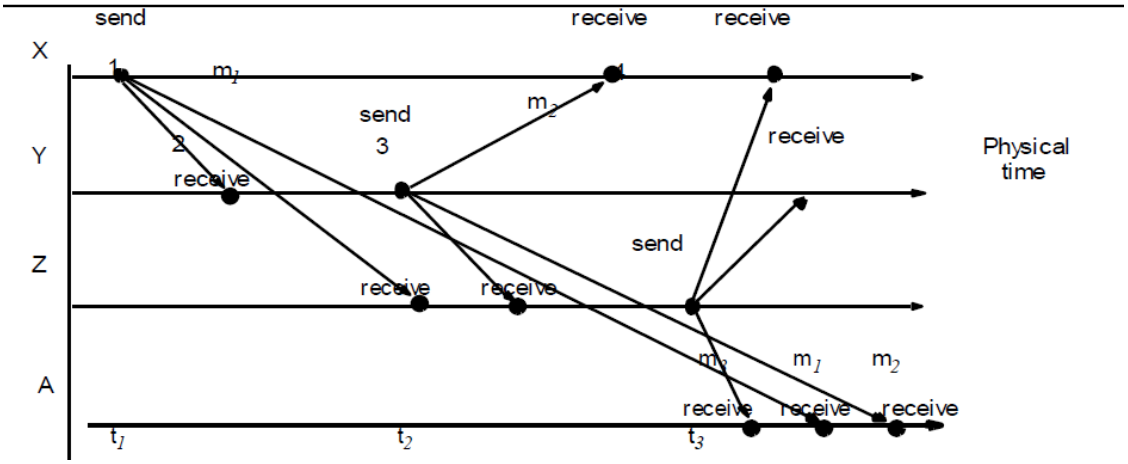

Fig 2.6: Ordering of events

1.6.3.2 Failure Model

In a DS, both processes and communication channels may fail. The following are the common types of failures:

- Omission Failure
- Arbitrary Failure
- Timing Failure

a) Omission failure

Omission failures occur due to communication link failures. They are detected through timeouts. They are classified as:

Process omission failures

- Omission failures that occur due to process crash (i.e.) the execution of the process could not continue. This is detected using timeouts.
- A period of time is fixed for all the methods to complete its execution. If the method takes time longer than the allowed time, a time out has occurred.
- In an asynchronous system a timeout can indicate only that a process is not responding.
- A process crash is called **fail-stop** if other processes can detect certainly that the process has crashed.
- Fail-stop behavior can be produced in a synchronous system if the processes use timeouts to detect when other processes fail to respond and messages are guaranteed to be delivered.

Communication omission failures

- This occurs when the messages are dropped between sender and the receiver. The message can be lost in sender buffer, receiver buffer or even in the communication channel.

- The loss of messages between the sending process and the outgoing message buffer is known as send omission failures. The loss of messages between the incoming message buffer and the receiving process as receive-omission failures.
- The loss of messages in a channel is channel omission failure.

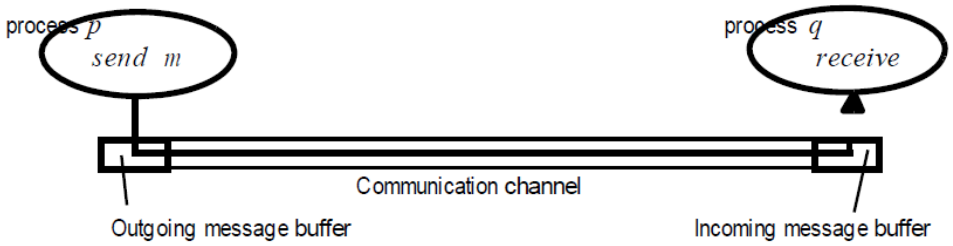

Fig 2.7: Communication between processes in a channel

b) Arbitrary failures (Byzantine failure):

This includes all possible errors that could cause failure. Communication channels often suffer from arbitrary failures. The following table gives an overview of the different failures and its categories:

Class	Affects	Comments
Fail stop	Process	Process halts and remains halted. Other processes may detect that this process has halted.
Crash	Process	Process halts and remains halted. Other processes may not detect that this process has halted.
Omission	Channel	A message inserted in an outgoing message buffer never arrives at the other end‟s incoming messagebuffer.
Send-omission	Process	A process completes a send operation but the message is not put in its outgoing message buffer.
Receive omission	Process	A message is put in a process‟s incoming message buffer, but that process does not receive it.
Arbitrary	Process or channel	Process/channel exhibits arbitrary behaviour: it may send/transmit arbitrary messages at arbitrary times or commit omissions; a process may stop or take an incorrect step.

c) Timing failures

- Timing failures refer to a situation where the environment in which a system operates does not behave as expected regarding the timing assumptions, that is, the timing constraints are not met.
- Timing failures are applicable in synchronous distributed system where time limits are set on process execution time, message delivery time and clock drift rate.
- The following are the common failures with respect to timings:

Class	Affects	Comments
Clock	Process	Process‟s local clock exceeds the bounds on its rate of drift from real time
Performance	Process	Process exceeds the bounds on the interval between two steps
Performance	Channel	A message‟s transmission takes longer than the stated bound.

Masking failures

- Each component in a distributed system is generally constructed from a collection of other components. It is always possible to construct reliable services from components that exhibit failures.
- A knowledge of failure characteristics of a component can enable a new service to be designed to mask the failure of the components on which it depends.

Reliability of one-to-one communication

The term reliable communication is defined in terms of validity and integrity.

Validity: Any message in the outgoing message buffer is eventually delivered to the incoming message buffer.

Integrity: The message received is identical to one sent, and no messages are delivered twice.

The threats to integrity come from two independent sources:

- Any protocol that retransmits messages but does not reject a message that arrives twice.
- Malicious users that may inject spurious messages, replay old messages or tamper with messages.

1.6.3.3 Security model

The security of a DS can be achieved by securing the processes and the channels used in their interactions and by protecting the objects that they encapsulate against unauthorized access. Protection is described in terms of objects, although the concepts apply equally well to resources of all types.

Protecting objects

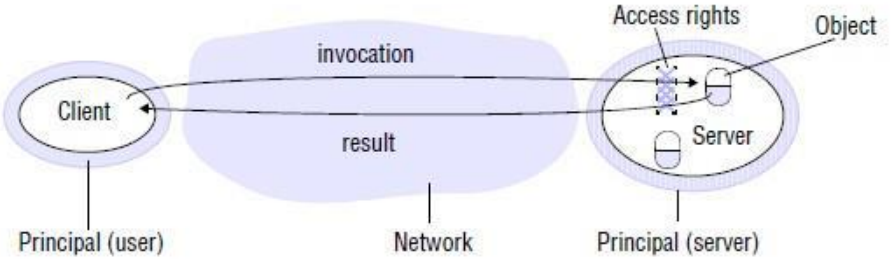

Fig 2.8: Objects and Principals

- The server manages a collection of objects on behalf of some users. The users can run client programs that send invocations to the server to perform operations on the objects.
- The server carries out the operation specified in each invocation and sends the result to the client.
- This uses use "access rights" that define who is allowed to perform operation on an object.
- The server should verify the identity of the principal (user) behind each operation and checking that they have sufficient access rights to perform the requested operation on the particular object, rejecting those who do not.
- A principal is an authority that manages the access rights. The principal may be a user or a process.

Securing processes and their interactions

- The messages send by the processes are exposed to attack because the network and the communication service that they use are open.
- To model security threats, we postulate an enemy that is capable of sending any process or reading/copying message between a pair of processes
- Threats form a potential enemy includes threats to processes, threats to communication

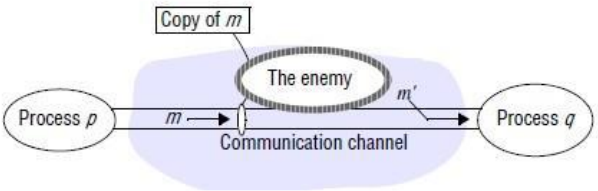

Fig: 2.9: Enemy

Threats to processes:

- A process that is designed to handle incoming requests may receive a message from any other process in the distributed system.It cannot determine the identity of the sender.
- This lack of reliable knowledge of the source of a message is a threat to the correct functioning of both servers and clients.

Servers:

- Servers cannot necessarily determine the identity of the principal behind any particular invocation. Without reliable knowledge of the sender"s identity, a server cannot tell whether to perform the operation or to reject it.

Clients:

- When a client receives the result of an invocation from a server, it cannot necessarily tell whether the source of the result message is from the intended server or from an enemy, perhaps „spoofing" the mail server.
- Thus the client could receive a result that was unrelated to the original invocation, such as a false mail item (one that is not in the user"s mailbox).

Threats to communication channels:

An enemy can copy, alter or inject messages as they travel across the network and its intervening gateways. Such attacks present a threat to the privacy and integrity of information as it travels over the network and to the integrity of the system.

Defeating security threats

Encryption and authentication are use to build secure channels. Each of the processes knows the identity of the principal on whose behalf the other process is executing and can check their access rights before performing an operation.